

Tag 3 - Hamming-Codes: Wir bauen den ersten echten fehlerkorrigierenden Code

Heute baust du selbst einen Code, der Fehler nicht nur findet, sondern repariert. Mit Bleistift, drei überlappenden Kreisen und Python. Am Ende kannst du **4 Datenbits in 7 Bits codieren**, und dein Decoder zeigt dir nicht nur, *dass* ein Fehler passiert ist, sondern **an welcher Position** – und korrigiert ihn automatisch.

Lernziele für heute

Am Ende von Tag 3 kannst du ...

- die **Hamming-Schranke** ableiten und damit ausrechnen, wie viele Prüfbits du für k Datenbits brauchst
- den **(7,4)-Hamming-Code** mit drei überlappenden Kreisen konstruieren
- erklären, warum die binäre Position eines Bits darüber entscheidet, von welchen Prüfbits es überwacht wird
- ein 4-Bit-Datenwort in 7 Bits **codieren** und ein 7-Bit-Codewort **dekodieren**
- bei einem Bitfehler die fehlerhafte Position **berechnen** und korrigieren
- den Code in Python implementieren und systematisch testen

Material: Bleistift, kariertes Papier, Buntstifte (drei verschiedene Farben sind hilfreich für die Kreise!), Thonny.

Block 1 · Wie viele Prüfbits brauchen wir mindestens? (≈ 30 min)

Erinnere dich an die Knobelaufgabe vom Vortag: *Du hast 4 Datenbits. Wie viele Prüfbits brauchst du mindestens, damit du jeden Einzel-Bitfehler korrigieren kannst?*

Lass uns das systematisch angehen.

Die Idee: jedes Codewort braucht "Platz" um sich herum

Bei n Bits insgesamt gibt es 2^n mögliche Wörter. Jedes davon kann entweder ein gültiges Codewort sein oder ein "verzerrtes" Codewort (nach einem Fehler).

Wir wollen, dass jedes empfangene 7-Bit-Wort eindeutig zu **einem** Codewort gehört – auch wenn unterwegs ein Bit gekippt ist. Das heißt:

- Jedes gültige Codewort belegt **eine Position** im Raum aller 2^n Wörter.
- Plus seine **n direkten Nachbarn** (für jeden möglichen Einzelfehler eine Position).
- Macht **$n + 1$ Positionen pro Codewort**, die ihm "gehören" müssen.

Bleistift-
Übung 1**Die Hamming-Schranke**

- (a) Wir haben 2^k Datenwörter (für $k = 4$ also 16 Stück), und jedes Codewort braucht $n+1$ Positionen für sich. Stelle die Ungleichung auf, die erfüllt sein muss, damit alle Codewörter und ihre Nachbarn nicht überlappen.

Tipp: 2^k Codewörter mal $(n+1)$ Positionen $\leq$ Gesamtzahl der Wörter.

- (b) Diese Ungleichung heißt **Hamming-Schranke** (engl. *Hamming bound*). Sie sagt: Mit weniger als so vielen Bits kann es prinzipiell **keinen** Code geben, der alle Einzelfehler korrigiert.
- (c) Fülle die Tabelle aus: Für jedes k von 1 bis 8 das kleinste n , das die Schranke erfüllt. Daraus ergibt sich die Anzahl Prüfbits $r = n - k$.

Datenbits k	Codelänge n (min.)	Prüfbits $r = n - k$
1		
2		
3		
4		
5		
6		
7		
8		

- (d) Was fällt dir auf? Bei welchen k “passt” die Schranke besonders gut (also ist die Ungleichung fast eine Gleichung)?

Das Ergebnis für $k = 4$

Wenn du die Tabelle richtig ausgefüllt hast, ist für $k = 4$ das minimale $n = 7$, also **3 Prüfbits**. Genauer: $2^4 \cdot (7+1) = 16 \cdot 8 = 128 = 2^7$. Die Hamming-Schranke ist **exakt** erreicht – kein einziges Bit zu viel!

Das ist der **(7,4)-Hamming-Code**, den wir heute bauen. Er ist ein “perfekter Code” in dem Sinn, dass er die Hamming-Schranke mit Gleichheit erreicht.

Wer war Hamming?

Richard Hamming arbeitete in den späten 1940ern an den Bell Labs an einem Computer mit Lochkarten. Seine Programme liefen über das Wochenende – und wenn auch nur eine Karte einen Lesefehler hatte, brach das ganze Programm ab. Der Computer **erkannte** den Fehler (mit einer einfachen Parität pro Spalte), aber er konnte ihn nicht **korrigieren**. Hamming musste dann am Montagmorgen von Hand alles neu starten.

Der berühmte Hamming-Ausspruch dazu: „Wenn die Maschine den Fehler erkennen kann – warum kann sie ihn dann nicht auch korrigieren?“ Aus dieser Frustration entstand 1950 sein Code.

Block 2 · Den (7,4)-Hamming-Code selbst konstruieren (≈ 60 min)

Jetzt wird's konkret. Wir nummerieren die 7 Bits mit den Positionen **1 bis 7**:

Position: 1 2 3 4 5 6 7

Beachte: Wir starten bei 1, nicht bei 0. Das hat einen tiefen mathematischen Grund, der gleich klar wird.

Wer ist Datenbit, wer Prüfbit?

Hamming hatte die geniale Idee, die Prüfbits an die Positionen zu setzen, die Zweierpotenzen sind: Position 1 ($= 2^0$), Position 2 ($= 2^1$) und Position 4 ($= 2^2$). Die übrigen Positionen 3, 5, 6, 7 bekommen die Datenbits.

Position:	1	2	3	4	5	6	7
Rolle:	p1	p2	d1	p3	d2	d3	d4
	↑	↑		↑			

Prüfbits an Zweierpotenz-Positionen

Welches Prüfbit überwacht welche Positionen?

Jetzt kommt der eigentliche Trick. Schreibe jede Position **als Binärzahl** auf:

Position	Binär	“passt zu” Prüfbit ...
1	001	p1
2	010	p2
3	011	p1, p2
4	100	p3
5	101	p1, p3
6	110	p2, p3
7	111	p1, p2, p3

Regel: Prüfbit p_j überwacht alle Positionen, deren Binärdarstellung an der j -ten Stelle eine 1 hat.

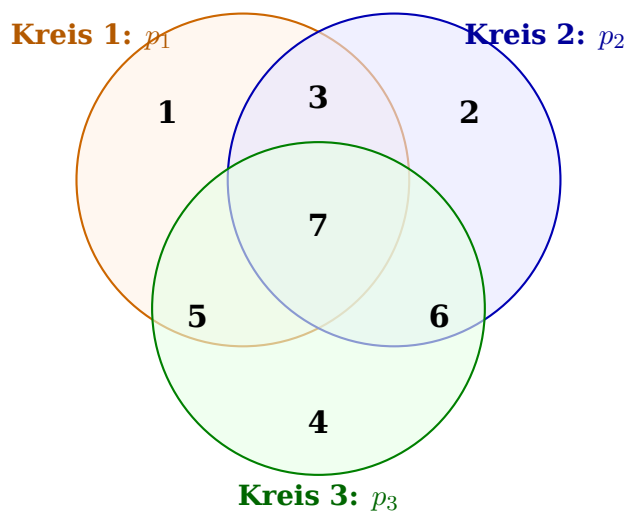
- p1 (steht selbst auf Position 1 = 001) überwacht alle Positionen mit ***
***1 → Positionen 1, 3, 5, 7

- p2 (steht selbst auf Position 2 = 010) überwacht alle Positionen mit ***
**1* → Positionen 2, 3, 6, 7
- p3 (steht selbst auf Position 4 = 100) überwacht alle Positionen mit ***
*1** → Positionen 4, 5, 6, 7

Jedes Prüfbit wird so gesetzt, dass die **Anzahl der Einsen in seiner Gruppe gerade** ist (also gerade Parität wie an Tag 1). Beachte: Das Prüfbit selbst gehört zu seiner Gruppe!

Die drei magischen Kreise

Wenn man das aufzeichnet, ergibt sich das berühmte Hamming-Venn-Diagramm: drei sich überlappende Kreise, jeder steht für ein Prüfbit und enthält die Positionen, die er kontrolliert.



Übertrag das auf ein eigenes Karoblatt – am besten mit drei verschiedenen Farben, eine pro Kreis. Markiere dir, welche Position in welchen Kreisen liegt. Position 7 zum Beispiel liegt in *allen drei* Kreisen, Position 4 nur in Kreis 3, und so weiter.

Codewort von Hand bauen

Datenwort: **d1 d2 d3 d4 = 1 0 1 1** (das ist der Buchstabe „K“ in 4-Bit-ASCII, naja, fast).

(a) Setze die Datenbits in die richtigen Positionen ein:

Position:	1	2	3	4	5	6	7
Inhalt:	?	?	1	?	0	1	1
	↑	↑		↑			
	p1	p2		p3			

(b) Berechne **p1**: Welche Positionen kontrolliert p1 (siehe Tabelle oben)? Trage den Inhalt dieser Positionen in deine Skizze ein und bestimme p1 so, dass die Gesamtzahl der Einsen in dieser Gruppe **gerade** ist.



Bleistift-
Übung 2

- (c) Berechne **p2** analog.
 (d) Berechne **p3** analog.
 (e) Schreibe das vollständige Codewort auf:

Position:	1	2	3	4	5	6	7
Codewort:	___	___	1	___	0	1	1

Mehr Codewörter

Bestimme die Codewörter für folgende Datenwörter (genauso wie eben):

d1	d2	d3	d4	p1	p2	p3	Codewort (1-7)
0	0	0	0				
1	1	1	1				
1	0	0	0				
0	1	1	0				

(Tipp: bei 0000 ist es einfach ;-). Bei 1111 müssen alle Prüfbits angepasst werden.)

Decodieren - der wirklich coole Trick

So weit so gut. Aber der eigentliche Zauber kommt jetzt: **Wie korrigieren wir Fehler?**

Stell dir vor, das Codewort 0110011 (das hattest du in Übung 2 für 1011) wird übertragen. Unterwegs kippt ein Bit. Du empfangst zum Beispiel 0010011 (Bit 2 ist gekippt).

Der Empfänger berechnet erneut die drei Prüfsummen:

- **s1**: Anzahl Einsen in Positionen 1, 3, 5, 7 $\rightarrow \text{mod } 2$
- **s2**: Anzahl Einsen in Positionen 2, 3, 6, 7 $\rightarrow \text{mod } 2$
- **s3**: Anzahl Einsen in Positionen 4, 5, 6, 7 $\rightarrow \text{mod } 2$

Bei einem fehlerfreien Codewort sind alle drei s gleich 0. Bei einem Fehler an Position p ist genau das s ungleich 0, dessen Prüfbit die Position p überwacht.

Hier ist der Knaller: Schreibe $s_3 s_2 s_1$ als **Binärzahl** auf. Diese Zahl ist genau **die Position des Fehlers!**

Decodieren von Hand

- (a) Empfangen: 0010011. Berechne s_1 , s_2 , s_3 (gerade Parität, jedes s ist 0 oder 1).

Nimm die Tabelle der Positionen oben zur Hilfe.

- (b) Schreibe $s_3 s_2 s_1$ als Binärzahl. In welcher Position ist der Fehler?



Bleistift-
Übung 3



Bleistift-
Übung 4

- (c) Korrigiere das empfangene Wort. Welches Datenwort wurde ursprünglich gesendet?
- (d) Probier es mit einem zweiten Beispiel: Empfangen 1100110. Wo ist der Fehler? Wie lautet das ursprüngliche Datenwort?

Warum funktioniert das?

Das ist die schönste Idee am Hamming-Code – und sie folgt direkt aus der binären Nummerierung der Positionen:

- Wenn das Bit an Position p gekippt ist, dann sind genau die Prüfsummen s_j „kaputt“, deren Prüfbit Position p überwacht.
- Welche Prüfbits überwachen Position p ? Genau die p_j , für die das j -te Bit der Binärdarstellung von p eine 1 ist.
- Also ist $s_3 s_2 s_1$ (binär gelesen) **genau** die Binärdarstellung von p !

Das ist keine Koinzidenz, sondern das Ergebnis von Hamming's genialer Wahl der Prüfbit-Positionen.

Block 3 · Python-Werkstatt: Encoder und Decoder (≈ 60 min)

Jetzt bauen wir das in Python. Lege in Thonny eine neue Datei hamming74.py an.

Encoder



Python-
Einheit 1

```
def hamming74_encode(daten):
    """Codiert 4 Datenbits in 7 Bits Hamming-Code.
    daten ist eine Liste [d1, d2, d3, d4].
    Rückgabe: Liste [bit1, bit2, ..., bit7]."""
    d1, d2, d3, d4 = daten

    # Prüfbits berechnen (gerade Parität in jeder Gruppe)
    p1 = (d1 + d2 + d4) % 2 # überwacht Positionen 3, 5, 7 (= d1, d2, d4)
    p2 = (d1 + d3 + d4) % 2 # überwacht Positionen 3, 6, 7 (= d1, d3, d4)
    p3 = (d2 + d3 + d4) % 2 # überwacht Positionen 5, 6, 7 (= d2, d3, d4)

    # In die richtige Reihenfolge einsetzen:
    # Position: 1 2 3 4 5 6 7
    return [p1, p2, d1, p3, d2, d3, d4]

# Test mit Datenwort 1011
print(hamming74_encode([1, 0, 1, 1]))
# Erwartet: [0, 1, 1, 0, 0, 1, 1] (das ist der Wert aus Bleistiftübung 2)
```

Decoder mit Fehlerkorrektur



Python-
Einheit 2

```
def hamming74_decode(empfangen):
    """Dekodiert 7 Bits Hamming-Code, korrigiert ggf. einen Einzelfehler.
    empfangen ist eine Liste [bit1, ..., bit7].
    Rückgabe: (daten, fehlerposition)
    daten = Liste der 4 Datenbits
    fehlerposition = 0 (kein Fehler) oder 1..7 (Position des Fehlers)"""
    b = empfangen.copy()

    # Prüfsummen berechnen
    s1 = (b[0] + b[2] + b[4] + b[6]) % 2 # Positionen 1, 3, 5, 7
    s2 = (b[1] + b[2] + b[5] + b[6]) % 2 # Positionen 2, 3, 6, 7
    s3 = (b[3] + b[4] + b[5] + b[6]) % 2 # Positionen 4, 5, 6, 7

    # s3 s2 s1 binär lesen → Fehlerposition (1..7) oder 0
    fehlerposition = s3 * 4 + s2 * 2 + s1

    # Fehler korrigieren falls nötig
    if fehlerposition != 0:
        b[fehlerposition - 1] = 1 - b[fehlerposition - 1]

    # Datenbits an den Positionen 3, 5, 6, 7 extrahieren
    daten = [b[2], b[4], b[5], b[6]]
    return daten, fehlerposition

# Test ohne Fehler:
codewort = hamming74_encode([1, 0, 1, 1])
print(hamming74_decode(codewort))
# Erwartet: ([1, 0, 1, 1], 0)

# Test mit Fehler an Position 5:
gestoert = codewort.copy()
gestoert[4] = 1 - gestoert[4] # Bit 5 (Index 4) kippen
print(hamming74_decode(gestoert))
# Erwartet: ([1, 0, 1, 1], 5)
```

Systematischer Test



Python-
Einheit 3

Jetzt der Lieblingsmoment: alle möglichen Einzelfehler durchsimulieren und prüfen, ob der Decoder wirklich alle korrigiert.

```
import itertools

def teste_alle_einzelfehler():
    """Testet alle 16 Datenwörter und alle 7 möglichen Einzelfehler."""
    erfolg = 0
    gesamt = 0
    for daten in itertools.product([0, 1], repeat=4):
        daten = list(daten)
        codewort = hamming74_encode(daten)
        for fehlerpos in range(7):
            gestoert = codewort.copy()
            gestoert[fehlerpos] = 1 - gestoert[fehlerpos]
            rekonstruiert, erkannte_pos = hamming74_decode(gestoert)
            gesamt += 1
            if rekonstruiert == daten:
                erfolg += 1
```

```

    else:
        print(f"FEHLER bei daten={daten}, fehlerpos={fehlerpos+1}: "
              f"rekonstruiert {rekonstruiert}, erkannt {erkannte_pos}")
    print(f"{erfolg}/{gesamt} Einzelfehler korrekt korrigiert")

teste_alle_einzelfehler()
# Erwartet: 112/112 (16 Datenwörter × 7 Fehlerpositionen)

```

Aufgabe 3.1 - Was passiert bei Doppelfehlern? Erweitere den Test, sodass er **zwei** Bits gleichzeitig kippt (an verschiedenen Positionen). Wie viele Doppelfehler werden trotzdem korrekt korrigiert? Wie viele werden falsch korrigiert (Decoder denkt, es war ein Einzelfehler an einer falschen Position)?

```

def teste_alle_doppelfehler():
    erfolg = 0
    falsch = 0
    gesamt = 0
    for daten in itertools.product([0, 1], repeat=4):
        daten = list(daten)
        codewort = hamming74_encode(daten)
        for pos1, pos2 in itertools.combinations(range(7), 2):
            gestoert = codewort.copy()
            gestoert[pos1] = 1 - gestoert[pos1]
            gestoert[pos2] = 1 - gestoert[pos2]
            rekonstruiert, _ = hamming74_decode(gestoert)
            gesamt += 1
            if rekonstruiert == daten:
                erfolg += 1
            else:
                falsch += 1
    print(f"Doppelfehler: {erfolg}/{gesamt} zufällig richtig, "
          f"{falsch}/{gesamt} falsch")

teste_alle_doppelfehler()

```

Du wirst sehen: **alle Doppelfehler werden falsch "korrigiert"**. Der Decoder denkt, es sei ein Einzelfehler, korrigiert das falsche Bit – und liefert ein falsches Datenwort. **Das ist gefährlich**, weil der Empfänger keinen Hinweis bekommt, dass etwas schief gelaufen ist!

Daher gibt es Erweiterungen wie **SECDED** (Single Error Correction, Double Error Detection): ein zusätzliches Paritätsbit erkennt Doppelfehler. Mehr dazu in den nächsten Tagen.

Block 4 · Wo steckt das überall drin? (≈ 15 min)

Was wir hier auf Karopapier bauen, ist die mathematische Grundlage für Verfahren, die heute jede Sekunde milliardenfach laufen.

ECC-RAM in Servern: Kosmische Strahlung (Höhenstrahlung, sogar in Bürogebäuden!) kippt gelegentlich Bits in Speicherchips. In normalem RAM bleibt das unbemerkt – im Server würde das ständig zu Abstürzen führen. Server-RAM nutzt daher SECDED-Codes auf Hamming-Basis: für jedes 64-Bit-Datenwort gibt es 8 Prüfbits.

Voyager-Sonden: Die beiden Voyager-Sonden, gestartet 1977, senden seit fast 50 Jahren Daten zur Erde – aus mittlerweile über 24 Milliarden Kilometern Entfernung. Bei dieser Entfernung ist das Signal so schwach, dass viele Bits unterwegs verloren gehen. Sie nutzen *verkettete Codes*: ein innerer Code (Hamming-artig) und ein äußerer Reed-Solomon-Code – genau das, was im Datamatrix-Symbol steckt.

WLAN, USB, SATA, Mobilfunk: Alle nutzen Fehlerkorrektur. Jedes Mal, wenn dein Handy ein WLAN-Paket empfängt, läuft im Hintergrund ein Decoder – auf Basis derselben Mathematik, die du heute mit Bleistift skizzierst.

Magnetische Speicher (Festplatten, SSDs): SSDs nutzen LDPC-Codes (Low-Density Parity-Check), die als Verallgemeinerung von Hamming-Codes verstanden werden können. Festplatten benutzen Reed-Solomon plus weitere Verfahren.

Der Punkt: Was du heute durchgerechnet hast, ist keine akademische Spielerei. Es ist die Basis dafür, dass Computer und Kommunikation überhaupt zuverlässig funktionieren.

Block 5 · Reflexion und Brücke zu Datamatrix (≈ 20 min)

Was wir heute gelernt haben

- Die **Hamming-Schranke** sagt uns, wie viele Prüfbits *mindestens* nötig sind, um Einzelfehler korrigieren zu können.
- Der **(7,4)-Hamming-Code** erreicht diese Schranke exakt und ist damit *perfekt*.
- Hamming's geniale Idee: Prüfbits an Zweierpotenz-Positionen, jedes überwacht die Positionen, deren Binärdarstellung an der entsprechenden Stelle eine 1 hat.
- Die **drei Prüfsummen ergeben binär gelesen die Position des Fehlers** – das ist keine Magie, sondern Folge der Konstruktion.
- Hamming-Codes korrigieren **einen** Bitfehler perfekt, aber Doppelfehler werden falsch "korrigiert" → daher braucht man in der Praxis SECDED oder stärkere Codes.

Drei Fragen zum Mitnehmen

1. Wenn der Hamming-Code (7,4) Einzelfehler korrigiert, aber bei Doppelfehlern lügt – was würde passieren, wenn wir 4 Datenbits in 8 Bits codieren (also ein Prüfbit mehr)? Wäre das ein guter Tausch?

2. Stell dir einen kaputten Datamatrix-Code vor: Auf einem zerkratzten Versandetikett ist nicht ein einzelnes Bit weg, sondern ein ganzer **Bereich** (mehrere benachbarte Bits gleichzeitig). Würde Hamming-Code damit klarkommen?
3. Hamming-Code arbeitet mit einzelnen Bits. Ein Datamatrix-Code aber speichert Bytes (Buchstaben, Ziffern). Was würde es bedeuten, wenn wir mit *Bytes* statt *Bits* rechnen? Gibt es so etwas wie eine "Byte-Parität"?

Vorschau Tag 4

Morgen schauen wir uns an, was passiert, wenn Fehler **gehäuft** auftreten – das ist nämlich die Realität bei zerkratzten Etiketten und beschädigten CDs. Wir lernen **CRC** (Cyclic Redundancy Check) kennen, das in WLAN und Ethernet steckt, und wir verstehen den Begriff **Bündelfehler**. Außerdem treffen wir auf das Konzept des **Interleaving** – ein einfacher, aber genialer Trick, mit dem man Bündelfehler in Einzelfehler verwandeln kann.

Damit machen wir den ersten Schritt vom Bit-Denken zum Byte-Denken – und in ein paar Tagen sind wir dann bei der Mathematik, die Datamatrix tatsächlich nutzt: **endliche Körper** und **Reed-Solomon**.

Lösungen (erst nach eigenem Versuch ansehen!)

Bleistiftübung 1 - Hamming-Schranke

- (a) Ungleichung: $2^k \cdot (n + 1) \leq 2^n$, mit $n = k + r$.
 (b) Umgeformt für r : $2^r \geq k + r + 1$.
 (c) Tabelle:

k	n (min.)	r
1	3	2
2	5	3
3	6	3
4	7	3
5	9	4
6	10	4
7	11	4
8	12	4

Rechenweg z. B. für $k = 4$: probiere $r = 1, 2, 3, \dots$ und prüfe, ob $2^r \geq k + r + 1$:
 - $r = 2$: $2^2 = 4$, aber $k + r + 1 = 7 \rightarrow$ reicht nicht
 - $r = 3$: $2^3 = 8$, $k + r + 1 = 8 \rightarrow$ passt **genau**

- (d) Besonders gut "passt" die Schranke bei $k = 1$ ($n = 3 \rightarrow 2^1 \cdot 4 = 8 = 2^3$), $k = 4$ ($n = 7 \rightarrow 2^4 \cdot 8 = 128 = 2^7$), $k = 11$ ($n = 15$), ... Diese sind die "perfekten" Hamming-Codes.

Bleistiftübung 2 - Codewort 1011

Datenbits: $d_1=1, d_2=0, d_3=1, d_4=1$, also Positionen 3, 5, 6, 7 mit Werten 1, 0, 1, 1.

Berechnung der Prüfbits (gerade Parität):

- p_1 überwacht Positionen 1, 3, 5, 7 = $p_1, 1, 0, 1 \rightarrow$ Summe der Daten $1+0+1 = 2$, also $p_1 = \mathbf{0}$ (damit Gesamtsumme gerade).
- p_2 überwacht Positionen 2, 3, 6, 7 = $p_2, 1, 1, 1 \rightarrow$ Summe $1+1+1 = 3$, also $p_2 = \mathbf{1}$.
- p_3 überwacht Positionen 4, 5, 6, 7 = $p_3, 0, 1, 1 \rightarrow$ Summe $0+1+1 = 2$, also $p_3 = \mathbf{0}$.

Codewort:

Position:	1	2	3	4	5	6	7
Codewort:	0	1	1	0	0	1	1

Bleistiftübung 3 - Mehr Codewörter

d1	d2	d3	d4	p1	p2	p3	Codewort (Position 1-7)
0	0	0	0	0	0	0	0 0 0 0 0 0 0
1	1	1	1	1	1	1	1 1 1 1 1 1 1
1	0	0	0	1	1	0	1 1 1 0 0 0 0
0	1	1	0	1	0	0	1 0 0 0 1 1 0

(Die ersten zwei sind besonders schön: das Null-Codewort und das Eins-Codewort. Letzteres hat Hamming-Distanz **7** zum ersten – der Code hat also Mindestdistanz mindestens 7? Nein, das ist nicht die Mindestdistanz; die ist 3. Andere Paare können kleinere Distanzen haben. Greta darf das gerne überprüfen – lass dafür alle 16 Codewörter generieren und finde die kleinste paarweise Hamming-Distanz.)

Bleistiftübung 4 - Decodieren

(a) Empfangen: 0 0 1 0 0 1 1.

- $s_1 = b_1 + b_3 + b_5 + b_7 \bmod 2 = 0 + 1 + 0 + 1 \bmod 2 = 2 \bmod 2 = \mathbf{0}$
- $s_2 = b_2 + b_3 + b_6 + b_7 \bmod 2 = 0 + 1 + 1 + 1 \bmod 2 = 3 \bmod 2 = \mathbf{1}$
- $s_3 = b_4 + b_5 + b_6 + b_7 \bmod 2 = 0 + 0 + 1 + 1 \bmod 2 = 2 \bmod 2 = \mathbf{0}$

(b) $s_3 s_2 s_1 = 0 1 0$ binär = **2** dezimal. Fehler an Position **2**.

(c) Korrigiert: Position 2 kippen $\rightarrow$ 0 1 1 0 0 1 1. Datenbits an Positionen 3, 5, 6, 7 = 1, 0, 1, 1. Ursprüngliches Datenwort: **1 0 1 1**. □

(d) Empfangen: 1 1 0 0 1 1 0.

- $s_1 = 1 + 0 + 1 + 0 = 2 \bmod 2 = \mathbf{0}$
- $s_2 = 1 + 0 + 1 + 0 = 2 \bmod 2 = \mathbf{0}$
- $s_3 = 0 + 1 + 1 + 0 = 2 \bmod 2 = \mathbf{0}$

Alle Prüfsummen 0 $\rightarrow$ **kein Fehler erkannt**. Datenbits = Positionen 3, 5, 6, 7 = 0, 1, 1, 0. Datenwort: **0 1 1 0**.

(Gemein: kein Fehler! Hat Greta das gemerkt?)

Aufgabe 3.1 - Doppelfehler-Test

Erwartete Ausgabe (sinngemäß):

Doppelfehler: 0/336 zufällig richtig, 336/336 falsch

Erklärung: Bei einem Doppelfehler ergeben die drei Prüfsummen genau die XOR-Summe der zwei gekippten Positionen (binär). Diese ist nie 0 (außer bei zwei *gleichen* Positionen, was nicht zählt) und entspricht einer **gültigen Position 1-7** – also wird der Decoder einen *dritten*, falschen Bit kippen. Resultat: 3 Bits gekippt insgesamt → Datenwort fast immer falsch.

Genau das motiviert SECDED: ein achttes Bit als “Gesamt-Parität” über alle 7 Bits erkennt zuverlässig, ob die Anzahl der Fehler ungerade (1) oder gerade (0, 2, 4, ...) ist. Damit lässt sich zwischen Einzel- und Doppelfehler unterscheiden.

Antworten zu den drei Reflexionsfragen

1. **8 statt 7 Bits:** Ja, das ist genau der **erweiterte Hamming-Code (8,4)** oder allgemein **SECDED**. Mit einem zusätzlichen Paritätsbit kann man Doppelfehler *erkennen* (aber immer noch nicht korrigieren). In ECC-RAM sind das (72, 64) – 64 Datenbits, 8 Prüfbits, korrigiert 1 Fehler, erkennt 2.
2. **Bündelfehler:** Hamming-Code hilft hier nicht. Wenn 2 oder mehr benachbarte Bits gleichzeitig kippen, wird der Code falsch “korrigieren”. Die Lösung: entweder die Daten **verteilen** (Interleaving), sodass benachbarte Übertragungsfehler nicht zu benachbarten Codebits werden – oder **stärkere Codes** mit größerer Korrekturkraft, z. B. Reed-Solomon.
3. **Byte-Parität:** Ja, das ist genau die Idee von Reed-Solomon. Statt mit Bits (0/1) rechnet man mit Bytes (0-255), und die “Modulo-Arithmetik” wird durch *Polynomarithmetik in einem endlichen Körper* ersetzt. Damit kann man pro Prüfbyte einen Bytefehler korrigieren – und ein einziger Bytefehler kann bis zu 8 Bitfehler abdecken (wenn alle in dasselbe Byte fallen). Genau diese Eigenschaft macht Reed-Solomon ideal für Bündelfehler.